

# Detección de Polvo en Paneles Solares mediante Redes Neuronales: Estudio Comparativo de Arquitecturas Transfer Learning

Ángel Manuel Pereira Rodríguez

19 de febrero de 2026

## Resumen

La energía solar es una de las fuentes de energía renovable más prometedoras para combatir el cambio climático. Sin embargo, la acumulación de polvo en paneles solares puede reducir su eficiencia energética entre un 15 % y 40 %, representando pérdidas económicas significativas. Este estudio presenta una comparación exhaustiva de siete arquitecturas de redes neuronales para la detección automática de polvo en paneles solares mediante clasificación binaria de imágenes (limpios vs. sucios). Se evaluaron tres modelos baseline —Red Neuronal Densa (DNN) con 77M parámetros, Red Neuronal Convolucional (CNN) con 7.9M parámetros, y una CNN personalizada— junto con cuatro arquitecturas de Transfer Learning: MobileNetV2, VGG16, ResNet50 y EfficientNetB0. Utilizando el dataset Solar Panel Dust Detection de Kaggle (~2,000 imágenes) y aplicando una estrategia de dos fases (Transfer Learning + Fine-Tuning), los modelos de Transfer Learning alcanzaron accuracies entre 84.41 % (MobileNetV2) y 88.89 % (ResNet50) tras fine-tuning, demostrando clara superioridad sobre los modelos entrenados desde cero (DNN: 70.71 %, CNN: 79.24 %). ResNet50 logró el mejor desempeño con 88.89 % accuracy, mientras que EfficientNetB0 destaca por su ratio accuracy/eficiencia con 85.19 % y solo 2.2M parámetros entrenables. Este estudio establece que Transfer Learning con estrategia de dos fases (congelamiento inicial + fine-tuning selectivo) es altamente efectivo para detección de polvo en paneles solares, proporcionando recomendaciones específicas para diferentes escenarios de implementación industrial.

## Índice

|                                               |          |
|-----------------------------------------------|----------|
| <b>1. Introducción</b>                        | <b>4</b> |
| 1.1. Contexto Energético y Problemática       | 4        |
| 1.2. Importancia del Mantenimiento Predictivo | 4        |
| 1.3. Estado del Arte en Detección Automática  | 5        |
| 1.4. Objetivos del Estudio                    | 5        |
| 1.5. Diseño Experimental                      | 5        |
| 1.6. Estructura del Documento                 | 6        |
| <b>2. Metodología</b>                         | <b>6</b> |
| 2.1. Descripción del Dataset                  | 6        |
| 2.1.1. Origen y Características               | 6        |
| 2.1.2. División de Datos                      | 6        |
| 2.1.3. Preprocesamiento de Datos              | 7        |
| 2.2. Arquitecturas de Modelos Baseline        | 7        |
| 2.2.1. Modelo 1: Red Neuronal Densa (DNN)     | 7        |

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| 2.2.2.    | Modelo 2: Red Neuronal Convolutacional (CNN)        | 8         |
| 2.3.      | Arquitecturas Transfer Learning                     | 9         |
| 2.3.1.    | Estrategia de Entrenamiento en Dos Fases            | 9         |
| 2.3.2.    | Capas Superiores Personalizadas (Común a Todos)     | 9         |
| 2.3.3.    | Modelo 3A: MobileNetV2                              | 10        |
| 2.3.4.    | Modelo 3B: VGG16                                    | 10        |
| 2.3.5.    | Modelo 3C: ResNet50                                 | 11        |
| 2.3.6.    | Modelo 3D: EfficientNetB0                           | 11        |
| 2.4.      | Configuración de Entrenamiento                      | 12        |
| 2.4.1.    | Hiperparámetros Fase 1 (Entrenamiento Inicial)      | 12        |
| 2.4.2.    | Hiperparámetros Fase 2 (Fine-Tuning)                | 12        |
| 2.4.3.    | Callbacks Implementados                             | 13        |
| 2.4.4.    | Data Augmentation                                   | 13        |
| 2.4.5.    | Hardware y Software                                 | 14        |
| <b>3.</b> | <b>Resultados</b>                                   | <b>14</b> |
| 3.1.      | Rendimiento de Modelos Baseline                     | 14        |
| 3.1.1.    | Resultados DNN (Red Densa)                          | 14        |
| 3.1.2.    | Resultados CNN (desde cero)                         | 14        |
| 3.2.      | Resultados Transfer Learning - Fase 1               | 15        |
| 3.3.      | Resultados Transfer Learning - Fase 2 (Fine-Tuning) | 15        |
| 3.4.      | Análisis Comparativo Global                         | 16        |
| 3.4.1.    | Ranking Final por Accuracy                          | 16        |
| 3.4.2.    | Análisis de Eficiencia                              | 16        |
| 3.4.3.    | Análisis de Velocidad de Inferencia                 | 17        |
| 3.4.4.    | Análisis de Matrices de Confusión                   | 17        |
| <b>4.</b> | <b>Discusión</b>                                    | <b>18</b> |
| 4.1.      | Superioridad Demostrada de Transfer Learning        | 18        |
| 4.1.1.    | Explicación de la Ventaja                           | 18        |
| 4.2.      | Comparación Entre Arquitecturas Transfer Learning   | 19        |
| 4.2.1.    | EfficientNetB0: El Campeón de Eficiencia            | 19        |
| 4.2.2.    | MobileNetV2: Rey del Edge Computing                 | 19        |
| 4.2.3.    | ResNet50: El Líder Absoluto en Accuracy             | 20        |
| 4.2.4.    | VGG16: El Veterano Sorprendente                     | 20        |
| 4.3.      | Importancia Crítica del Fine-Tuning                 | 21        |
| 4.3.1.    | Ganancia Consistente                                | 21        |
| 4.3.2.    | Mecanismo del Fine-Tuning                           | 21        |
| 4.3.3.    | Best Practices Identificadas                        | 21        |
| 4.4.      | Implicaciones Prácticas para Industria Solar        | 21        |
| 4.4.1.    | Reducción de Costos de Mantenimiento                | 21        |
| 4.4.2.    | Escenarios de Despliegue                            | 22        |
| 4.4.3.    | Integración con Sistemas SCADA                      | 22        |
| 4.5.      | Limitaciones del Estudio                            | 23        |
| 4.5.1.    | Limitaciones del Dataset                            | 23        |
| 4.5.2.    | Limitaciones Metodológicas                          | 23        |
| 4.5.3.    | Consideraciones de Despliegue Real                  | 23        |
| 4.6.      | Generalizabilidad de Resultados                     | 24        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>5. Conclusiones</b>                              | <b>24</b> |
| 5.1. Hallazgos Principales                          | 24        |
| 5.2. Recomendaciones para Implementación Industrial | 25        |
| 5.2.1. Selección de Modelo por Escenario            | 25        |
| 5.2.2. Pipeline de Implementación Recomendado       | 25        |
| 5.2.3. Mejores Prácticas Establecidas               | 26        |
| 5.3. Trabajos Futuros                               | 26        |
| 5.3.1. Extensiones Arquitectónicas                  | 26        |
| 5.3.2. Extensión del Problema                       | 27        |
| 5.3.3. Validación y Robustez                        | 27        |
| 5.3.4. Explainability e Interpretabilidad           | 28        |
| 5.3.5. Optimización de Despliegue                   | 28        |
| 5.4. Impacto Esperado                               | 28        |
| 5.4.1. Impacto Ambiental                            | 28        |
| 5.4.2. Impacto Económico                            | 29        |
| 5.4.3. Impacto Tecnológico                          | 29        |
| 5.5. Reflexión Final                                | 29        |

## Índice de cuadros

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| 1. Distribución estimada del dataset por conjunto                              | 7  |
| 2. Comparación de modelos baseline                                             | 9  |
| 3. Comparación de modelos Transfer Learning                                    | 12 |
| 4. Hiperparámetros Fase 1 - Todos los modelos                                  | 12 |
| 5. Hiperparámetros Fase 2 - Solo Transfer Learning                             | 12 |
| 6. Técnicas de data augmentation                                               | 13 |
| 7. Comparación de resultados baseline (valores reales promedio de 4 notebooks) | 15 |
| 8. Resultados Fase 1 - Transfer Learning (valores reales de test)              | 15 |
| 9. Resultados Fase 2 - Post Fine-Tuning (evaluación en test)                   | 16 |
| 10. Ranking final de los 7 modelos evaluados (Fase 2 TL+FT en test)            | 16 |
| 11. Análisis de eficiencia: parámetros por punto de accuracy                   | 17 |
| 12. Velocidad de inferencia estimada (valores típicos en GPU RTX 4060 Ti)      | 17 |
| 13. Matriz de recomendación modelo-escenario                                   | 25 |

# 1 Introducción

## 1.1 Contexto Energético y Problemática

La transición hacia fuentes de energía renovable se ha convertido en una prioridad global para mitigar el cambio climático y garantizar la sostenibilidad energética. Entre las tecnologías renovables, la energía solar fotovoltaica ha experimentado un crecimiento exponencial en las últimas dos décadas, convirtiéndose en una de las opciones más viables tanto para instalaciones residenciales como para grandes plantas de generación.

Sin embargo, la eficiencia de los paneles solares se ve significativamente afectada por factores ambientales, siendo la acumulación de polvo uno de los más críticos. Estudios muestran que la deposición de polvo, arena, polen y otros contaminantes puede reducir la eficiencia de conversión energética entre un 15 % y 40 %, dependiendo de:

- **Ubicación geográfica:** Regiones áridas y desérticas experimentan mayor acumulación
- **Condiciones climáticas:** Ausencia de lluvia natural para limpieza
- **Densidad de tráfico:** Proximidad a carreteras aumenta partículas suspendidas
- **Actividad industrial:** Emisiones de fábricas cercanas
- **Ángulo de inclinación:** Paneles horizontales acumulan más polvo

## 1.2 Importancia del Mantenimiento Predictivo

El mantenimiento tradicional de plantas solares se basa en inspecciones manuales periódicas o programas de limpieza fijos, enfoques que presentan limitaciones:

1. **Alto costo operativo:** Requiere personal especializado y equipamiento
2. **Ineficiencia:** Limpieza innecesaria de paneles limpios o tardía de paneles críticos
3. **No escalable:** Difícil de implementar en instalaciones de gran escala
4. **Riesgo humano:** Trabajo en alturas o condiciones adversas
5. **Subjetividad:** Evaluación visual dependiente de experiencia del inspector

La implementación de sistemas de detección automática mediante visión por computadora y aprendizaje profundo ofrece ventajas significativas:

- Detección temprana de acumulación crítica
- Optimización de calendarios de limpieza
- Reducción de costos operativos (60-80 %)
- Monitorización continua 24/7
- Integración con sistemas SCADA y drones
- Datos cuantitativos para análisis histórico

### 1.3 Estado del Arte en Detección Automática

Las aproximaciones tradicionales para detección de polvo incluyen:

- **Sensores de irradiancia:** Comparan entrada teórica vs. real, pero no localizan problema
- **Análisis de curvas IV:** Requiere equipamiento especializado
- **Termografía infrarroja:** Detecta hot spots pero no cuantifica polvo
- **Inspección manual:** Lenta, costosa, no escalable

El avance de técnicas de Deep Learning ha permitido desarrollar sistemas de visión por computadora capaces de clasificar automáticamente el estado de limpieza de paneles solares a partir de imágenes. Transfer Learning, en particular, ha demostrado ser altamente efectivo en tareas de clasificación de imágenes con datasets limitados, aprovechando conocimiento pre-aprendido de grandes corpus como ImageNet.

### 1.4 Objetivos del Estudio

Este trabajo presenta un estudio comparativo sistemático con los siguientes objetivos:

1. **Evaluar siete arquitecturas diferentes** de redes neuronales para clasificación binaria (panel limpio vs. panel sucio)
2. **Comparar enfoques baseline** (DNN, CNN desde cero) con Transfer Learning
3. **Analizar cuatro modelos pre-entrenados** populares: MobileNetV2, VGG16, ResNet50 y EfficientNetB0
4. **Determinar el trade-off** entre precisión, eficiencia computacional y velocidad de inferencia
5. **Proporcionar recomendaciones** específicas para diferentes escenarios de despliegue (cloud, edge, drones)
6. **Establecer mejores prácticas** para entrenamiento, data augmentation y fine-tuning

### 1.5 Diseño Experimental

El estudio sigue un diseño experimental controlado donde:

- **Variable constante:** Dataset, preprocesamiento, configuración de entrenamiento
- **Variable experimental:** Arquitectura del modelo (7 variantes)
- **Metodología:** Cuatro notebooks Jupyter implementan el mismo pipeline con diferentes modelos Transfer Learning
- **Evaluación:** Métricas estándar (accuracy, precision, recall, F1-score) + análisis de eficiencia

## 1.6 Estructura del Documento

Este documento está organizado de la siguiente manera:

- **Sección 2:** Metodología común a todos los experimentos (dataset, preprocesamiento, arquitecturas)
- **Sección 3:** Resultados individuales de modelos baseline y Transfer Learning
- **Sección 4:** Análisis comparativo global de los 7 modelos
- **Sección 5:** Discusión de hallazgos, implicaciones prácticas y limitaciones
- **Sección 6:** Conclusiones y recomendaciones para implementación industrial

## 2 Metodología

### 2.1 Descripción del Dataset

#### 2.1.1 Origen y Características

El dataset utilizado es el *Solar Panel Dust Detection* disponible públicamente en Kaggle<sup>1</sup>, creado específicamente para tareas de clasificación binaria de estado de limpieza de paneles solares.

**Características principales:**

- **Total de imágenes:** Aproximadamente 2,000 imágenes de alta resolución
- **Clases:** 2 (clasificación binaria)
  - **Clase 0 - Clean (Limpio):** Paneles sin acumulación significativa de polvo
  - **Clase 1 - Dusty (Sucio):** Paneles con acumulación visible de polvo/contaminantes
- **Distribución:** Dataset balanceado (~50 % por clase)
- **Fuente:** Imágenes recopiladas de instalaciones solares reales
- **Formato original:** JPEG de resolución variable

#### 2.1.2 División de Datos

Se aplicó una estrategia de división jerárquica para crear conjuntos de entrenamiento, validación y prueba:

1. **División inicial:** 80 % Training / 20 % Test
2. **Subdivisión de Training:** 80 % Training / 20 % Validation
3. **Resultado final:** 64 % Training / 16 % Validation / 20 % Test (aproximado)

La Tabla 1 muestra la distribución aproximada del dataset:

---

<sup>1</sup><https://www.kaggle.com/datasets/hemanthsai7/solar-panel-dust-detection>

Cuadro 1: Distribución estimada del dataset por conjunto

| Clase        | Training      | Validation  | Test        | Total  |
|--------------|---------------|-------------|-------------|--------|
| Clean        | ~640 (50 %)   | ~160 (50 %) | ~200 (50 %) | ~1,000 |
| Dusty        | ~640 (50 %)   | ~160 (50 %) | ~200 (50 %) | ~1,000 |
| <b>Total</b> | ~1,280 (64 %) | ~320 (16 %) | ~400 (20 %) | ~2,000 |

**Nota:** La división se realizó con estratificación para mantener la proporción de clases en todos los conjuntos.

### 2.1.3 Preprocesamiento de Datos

Todas las imágenes fueron sometidas al siguiente pipeline de preprocesamiento uniforme:

#### 1. Carga y conversión:

- Lectura con OpenCV (formato BGR)
- Conversión a RGB para compatibilidad con modelos ImageNet

#### 2. Redimensionamiento:

- Redimensión a  $224 \times 224$  píxeles (estándar ImageNet)
- Interpolación bilineal para preservar calidad

#### 3. Normalización (variable por modelo):

- **DNN/CNN baseline:** Escalado a  $[0, 1]$  (división por 255)
- **MobileNetV2:** Escalado a  $[0, 1]$
- **VGG16/ResNet50:** Normalización Caffe-style (sustracción de media ImageNet por canal RGB:  $[103.939, 116.779, 123.68]$ )
- **EfficientNetB0:** Escalado a  $[-1, 1]$

#### 4. Codificación de etiquetas:

- Conversión a formato one-hot encoding
- Clean:  $[1, 0]$ , Dusty:  $[0, 1]$

#### 5. Aleatorización:

- Barajado aleatorio con semilla fija (reproducibilidad)
- Aplicado a conjuntos de training y validation

## 2.2 Arquitecturas de Modelos Baseline

Se implementaron tres modelos baseline como referencia comparativa:

### 2.2.1 Modelo 1: Red Neuronal Densa (DNN)

Arquitectura completamente conectada que sirve como baseline simple:

**Estructura:**

- **Input Layer:** Imágenes aplanadas  $\rightarrow$  vector de 150,528 features ( $224 \times 224 \times 3$ )
- **Dense Block 1:** Dense(512) + ReLU + BatchNormalization + Dropout(0.5)

- **Dense Block 2:** Dense(256) + ReLU + BatchNormalization + Dropout(0.5)
- **Dense Block 3:** Dense(128) + ReLU + BatchNormalization + Dropout(0.5)
- **Output Layer:** Dense(2) + Softmax

**Características:**

- Parámetros entrenables:  $\sim 77$  millones
- Sin data augmentation (incompatible con input aplanado)
- Regularización: BatchNorm + Dropout agresivo (0.5)
- Limitación: No aprovecha estructura espacial de imágenes

## 2.2.2 Modelo 2: Red Neuronal Convolutiva (CNN)

CNN diseñada desde cero con arquitectura jerárquica:

**Estructura (4 bloques convolucionales):**

- **Conv Block 1:**
  - Conv2D(32,  $3 \times 3$ , padding='same') + ReLU
  - Conv2D(32,  $3 \times 3$ , padding='same') + ReLU
  - MaxPooling2D( $2 \times 2$ )
  - BatchNormalization
  - Dropout(0.25)
- **Conv Block 2:** Similar a Block 1, filtros=64
- **Conv Block 3:** Similar a Block 1, filtros=128
- **Conv Block 4:** Similar a Block 1, filtros=256
- **Dense Layers:**
  - Flatten
  - Dense(512) + ReLU + BatchNorm + Dropout(0.5)
  - Dense(256) + ReLU + BatchNorm + Dropout(0.5)
  - Dense(2) + Softmax

**Características:**

- Parámetros entrenables:  $\sim 7.9$  millones
- Reducción espacial progresiva:  $224 \times 224 \rightarrow 14 \times 14$
- Aumento de profundidad de features:  $3 \rightarrow 256$  canales
- Con data augmentation



Cuadro 2: Comparación de modelos baseline

| Característica        | DNN          | CNN                       |
|-----------------------|--------------|---------------------------|
| Parámetros            | 77M          | 7.9M                      |
| Input                 | Aplanado     | $224 \times 224 \times 3$ |
| Capas convolucionales | 0            | 8                         |
| Capas densas          | 3            | 2                         |
| Data augmentation     | No           | Sí                        |
| Regularización        | Dropout(0.5) | Dropout(0.25/0.5)         |

## 2.3 Arquitecturas Transfer Learning

Los cuatro modelos de Transfer Learning comparten una estrategia común de dos fases:

### 2.3.1 Estrategia de Entrenamiento en Dos Fases

#### Fase 1: Entrenamiento con Base Congelada

1. Cargar modelo pre-entrenado en ImageNet (sin capa superior)
2. Congelar todas las capas del modelo base (trainable=False)
3. Añadir capas personalizadas superiores
4. Entrenar solo las nuevas capas ( $\sim 50$  épocas)
5. Objetivo: Adaptar features de ImageNet al problema específico

#### Fase 2: Fine-Tuning Selectivo

1. Descongelar el 35 % superior de capas del modelo base
2. Mantener capas BatchNormalization congeladas (crítico para estabilidad)
3. Reducir learning rate drásticamente ( $1e-5$ )
4. Entrenar conjuntamente capas descongeladas + capas superiores ( $\sim 30$  épocas)
5. Objetivo: Ajuste fino de features de alto nivel al dominio solar

### 2.3.2 Capas Superior es Personalizadas (Común a Todos)

Todos los modelos Transfer Learning utilizan la misma arquitectura superior:

- **GlobalAveragePooling2D:** Reduce dimensionalidad espacial manteniendo información semántica
- **Dense(512) + ReLU:** Primera capa densa con regularización L2
- **BatchNormalization:** Normalización de activaciones
- **Dropout(0.6):** Regularización agresiva
- **Dense(256) + ReLU:** Segunda capa densa con regularización L2
- **BatchNormalization**
- **Dropout(0.6)**

- **Dense(2) + Softmax:** Capa de salida binaria

#### **Justificación de GlobalAveragePooling vs Flatten:**

- Reduce drásticamente parámetros (evita overfitting)
- Mantiene invarianza espacial
- Facilita interpretación (mapeo directo features → clases)
- Estándar en arquitecturas modernas

### **2.3.3 Modelo 3A: MobileNetV2**

#### **Características arquitectónicas:**

- **Origen:** Google (2018), diseñado para dispositivos móviles
- **Capas totales:** 154 capas en modelo base
- **Parámetros base:** ~3.5 millones
- **Innovación clave:** Bloques residuales invertidos con expansión lineal
- **Normalización:** Escalado [0, 1]

#### **Parámetros por fase:**

- Fase 1 (base congelada): ~1.4M entrenables, 2.3M congelados
- Fase 2 (fine-tuning 35 %): ~2.9M entrenables

#### **Ventajas:**

- Muy ligero y rápido (ideal para edge computing)
- Bajo consumo de memoria RAM
- Inferencia en tiempo real en CPU
- Excelente para despliegue en drones o dispositivos IoT

### **2.3.4 Modelo 3B: VGG16**

#### **Características arquitectónicas:**

- **Origen:** Oxford Visual Geometry Group (2014)
- **Capas totales:** 19 capas (16 convolucionales + 3 fully connected)
- **Parámetros base:** ~14.7M en parte convolucional (~138M total con FC)
- **Diseño:** Arquitectura secuencial simple (sin skip connections)
- **Normalización:** Caffe-style (restar media ImageNet RGB)

#### **Parámetros por fase:**

- Fase 1 (base congelada): ~1.6M entrenables, 14.7M congelados
- Fase 2 (fine-tuning 35 %): ~6.5M entrenables

**Características:**

- Arquitectura clásica y probada
- Fácil de entender e interpretar
- Entrenamiento muy estable
- Alto consumo de memoria (mayor modelo evaluado)
- Más lento en inferencia

**2.3.5 Modelo 3C: ResNet50****Características arquitectónicas:**

- **Origen:** Microsoft Research (2015), ganador ImageNet 2015
- **Capas totales:** 50 capas de profundidad
- **Parámetros base:** ~23.6 millones
- **Innovación clave:** Residual connections (skip connections) que mitigan vanishing gradient
- **Normalización:** Caffe-style

**Parámetros por fase:**

- Fase 1 (base congelada): ~1.4M entrenables, 23.6M congelados
- Fase 2 (fine-tuning 35 %): ~8.7M entrenables

**Ventajas:**

- Permite entrenar redes muy profundas sin degradación
- Excelente extractor de features jerárquico
- Balance entre rendimiento y eficiencia
- Skip connections facilitan propagación de gradientes

**2.3.6 Modelo 3D: EfficientNetB0****Características arquitectónicas:**

- **Origen:** Google (2019), estado del arte en eficiencia
- **Capas totales:** 237 capas
- **Parámetros base:** ~4.0 millones (el más eficiente)
- **Innovación clave:** Compound scaling (escala width, depth y resolution simultáneamente)
- **Normalización:** Escalado a  $[-1, 1]$

**Parámetros por fase:**

- Fase 1 (base congelada): ~1.3M entrenables, 4.0M congelados
- Fase 2 (fine-tuning 35 %): ~2.2M entrenables

**Ventajas:**

- **Mejor ratio accuracy/eficiencia** del mercado
- Arquitectura moderna optimizada mediante NAS (Neural Architecture Search)
- Escalable a diferentes tamaños (B0-B7)
- Excelente para producción (balance rendimiento/recursos)

Cuadro 3: Comparación de modelos Transfer Learning

| Modelo         | Params<br>Base | Capas | Fase 1<br>(entrenable) | Fase 2<br>(entrenable) |
|----------------|----------------|-------|------------------------|------------------------|
| MobileNetV2    | 3.5M           | 154   | 1.4M                   | 2.9M                   |
| VGG16          | 14.7M          | 19    | 1.6M                   | 6.5M                   |
| ResNet50       | 23.6M          | 50    | 1.4M                   | 8.7M                   |
| EfficientNetB0 | 4.0M           | 237   | 1.3M                   | 2.2M                   |

**2.4 Configuración de Entrenamiento****2.4.1 Hiperparámetros Fase 1 (Entrenamiento Inicial)**

Cuadro 4: Hiperparámetros Fase 1 - Todos los modelos

| Parámetro          | Valor                    |
|--------------------|--------------------------|
| Optimizador        | Adam                     |
| Learning rate      | 0.001                    |
| Función de pérdida | Categorical Crossentropy |
| Batch size         | 32                       |
| Épocas máximas     | 50                       |
| Métrica principal  | Accuracy                 |

**2.4.2 Hiperparámetros Fase 2 (Fine-Tuning)**

Cuadro 5: Hiperparámetros Fase 2 - Solo Transfer Learning

| Parámetro           | Valor                         |
|---------------------|-------------------------------|
| Learning rate       | 1e-5 (reducido 100×)          |
| Capas descongeladas | 35 % superior del modelo base |
| BatchNorm layers    | Mantener congeladas           |
| Épocas adicionales  | 30                            |
| Resto de parámetros | Igual que Fase 1              |

**Justificación de LR bajo en Fase 2:**

- Evita destruir features pre-aprendidas útiles
- Permite ajuste fino sin oscilaciones
- Critical para estabilidad de BatchNorm layers

### 2.4.3 Callbacks Implementados

#### 1. EarlyStopping:

- Monitor: `val_loss`
- Paciencia: 10 épocas
- Modo: Minimizar
- Restaurar mejores pesos: Activado
- Objetivo: Prevenir overfitting y ahorrar tiempo computacional

#### 2. ReduceLROnPlateau:

- Monitor: `val_loss`
- Factor de reducción: 0.5
- Paciencia: 5 épocas
- Learning rate mínimo:  $1e-7$
- Objetivo: Ajuste fino adaptativo cuando training se estanca

### 2.4.4 Data Augmentation

Aplicado a CNN y todos los modelos Transfer Learning (no a DNN por input aplanado):

Cuadro 6: Técnicas de data augmentation

| Transformación            | Parámetro            |
|---------------------------|----------------------|
| Rotación aleatoria        | $\pm 15-20$ grados   |
| Desplazamiento horizontal | 10-20 % del ancho    |
| Desplazamiento vertical   | 10-20 % del alto     |
| Volteo horizontal         | Probabilidad 0.5     |
| Volteo vertical           | Probabilidad 0.5     |
| Zoom aleatorio            | $\pm 15-20$ %        |
| Transformación de corte   | 10-15 %              |
| Modo de relleno           | <code>nearest</code> |

#### Justificación:

- Aumenta variabilidad artificial del dataset limitado
- Mejora generalización a condiciones de captura variables
- Simula diferentes ángulos de cámara/dron
- Reduce overfitting significativamente

### 2.4.5 Hardware y Software

#### Hardware:

- GPU: NVIDIA GeForce RTX 4060 Ti
- VRAM: 16 GB
- Configuración: Memory growth habilitado (asignación dinámica)

#### Software:

- Framework: TensorFlow 2.10.0 (con soporte CUDA)
- Keras: 2.10.0
- Python: 3.9
- CUDA Toolkit: Compatible con TF 2.10
- cuDNN: Habilitado
- NumPy: 1.26.4
- OpenCV: <4.10

## 3 Resultados

### 3.1 Rendimiento de Modelos Baseline

Los modelos DNN y CNN sirven como benchmarks para evaluar la mejora proporcionada por Transfer Learning.

#### 3.1.1 Resultados DNN (Red Densa)

El modelo DNN, a pesar de sus 77M parámetros, mostró limitaciones inherentes:

- **Test Accuracy:** 70.71 % (promedio de 4 notebooks)
- **Problema principal:** No captura estructura espacial de imágenes
- **Overfitting:** Alta brecha train-validation a pesar de Dropout(0.5)
- **Convergencia:** Lenta, requiere muchas épocas

#### 3.1.2 Resultados CNN (desde cero)

La CNN personalizada demostró mejora significativa sobre DNN:

- **Test Accuracy:** 79.24 % (promedio de 4 notebooks)
- **Ventaja:** Extracción jerárquica de features espaciales
- **Parámetros:** 10× menos que DNN (7.9M vs 77M)
- **Limitación:** Requiere más datos para competir con Transfer Learning

Cuadro 7: Comparación de resultados baseline (valores reales promedio de 4 notebooks)

| Modelo | Accuracy | Precision | Recall | F1     |
|--------|----------|-----------|--------|--------|
| DNN    | 70.71 %  | 0.7019    | 0.6884 | 0.6915 |
| CNN    | 79.24 %  | 0.7983    | 0.7774 | 0.7798 |

**Observaciones:**

- CNN supera a DNN en 8.53 puntos porcentuales
- Ambos modelos insuficientes para despliegue industrial (<80 % accuracy)
- DNN muestra alta variabilidad entre ejecuciones (68.62 %-72.71 %)
- CNN más estable y eficiente (con 10× menos parámetros que DNN)
- Justifica necesidad de Transfer Learning para superar el 80 %

**3.2 Resultados Transfer Learning - Fase 1**

Todos los modelos Transfer Learning superaron significativamente a los baselines en la Fase 1.

Cuadro 8: Resultados Fase 1 - Transfer Learning (valores reales de test)

| Modelo         | Test Acc | Precision | Recall | F1     |
|----------------|----------|-----------|--------|--------|
| MobileNetV2    | 79.34 %  | 0.8222    | 0.7630 | 0.7714 |
| VGG16          | 81.29 %  | 0.8074    | 0.8116 | 0.8090 |
| ResNet50       | 84.41 %  | 0.8581    | 0.8244 | 0.8332 |
| EfficientNetB0 | 84.21 %  | 0.8370    | 0.8406 | 0.8386 |

**Observaciones clave:**

1. **Salto de rendimiento:** +0.10 a +13.70 puntos sobre baselines
2. **ResNet50 líder absoluto:** 84.41 % accuracy (+5.17 puntos sobre CNN baseline)
3. **EfficientNetB0 muy cercano:** 84.21 % con solo 0.20 puntos menos que ResNet50
4. **VGG16 sólido:** 81.29 % superando a MobileNetV2 por 1.95 puntos
5. **MobileNetV2 competitivo:** 79.34 % con tamaño óptimo para edge devices
6. **Features pre-aprendidas efectivas:** ImageNet generaliza bien a paneles solares
7. **Balance Precision-Recall:** ResNet50 y EfficientNetB0 muestran mejor equilibrio

**3.3 Resultados Transfer Learning - Fase 2 (Fine-Tuning)**

El fine-tuning selectivo (descongelamiento del 35 % superior de capas) proporcionó mejoras significativas en todos los modelos evaluados en test.

Cuadro 9: Resultados Fase 2 - Post Fine-Tuning (evaluación en test)

| Modelo         | Fase 1<br>Acc Test | Fase 2<br>Acc Test | Delta<br>Absoluto | Mejora<br>Relativa |
|----------------|--------------------|--------------------|-------------------|--------------------|
| MobileNetV2    | 79.34 %            | 84.41 %            | +5.07 %           | +6.4 %             |
| VGG16          | 81.29 %            | 86.16 %            | +4.87 %           | +6.0 %             |
| ResNet50       | 84.41 %            | 88.89 %            | +4.48 %           | +5.3 %             |
| EfficientNetB0 | 84.21 %            | 85.19 %            | +0.97 %           | +1.2 %             |

**Hallazgos importantes:**

1. **Fine-tuning efectivo:** Todos los modelos mejoraron en test, validando la estrategia de dos fases
2. **ResNet50 líder absoluto:** 88.89 % accuracy (+4.48 % sobre Fase 1)
3. **MobileNetV2 mayor incremento:** +5.07 % de mejora relativa, alcanzando 84.41 %
4. **VGG16 sorprende:** 86.16 % a pesar de arquitectura antigua
5. **EfficientNetB0 menor mejora:** +0.97 %, posible convergencia temprana o arquitectura ya óptima
6. **Ausencia de overfitting:** Todas las mejoras se transfirieron al conjunto de test
7. **Todos superan 84 %:** Umbral de utilidad para aplicaciones industriales

**3.4 Análisis Comparativo Global****3.4.1 Ranking Final por Accuracy**

Cuadro 10: Ranking final de los 7 modelos evaluados (Fase 2 TL+FT en test)

| Pos | Modelo                 | Test Accuracy | Parámetros        |
|-----|------------------------|---------------|-------------------|
| 1   | ResNet50 (TL+FT)       | 88.89 %       | 8.7M (entrenable) |
| 2   | VGG16 (TL+FT)          | 86.16 %       | 6.5M (entrenable) |
| 3   | EfficientNetB0 (TL+FT) | 85.19 %       | 2.2M (entrenable) |
| 4   | MobileNetV2 (TL+FT)    | 84.41 %       | 2.9M (entrenable) |
| 5   | CNN (desde cero)       | 79.24 %       | 7.9M              |
| 6   | DNN (baseline)         | 70.71 %       | 77M               |

**3.4.2 Análisis de Eficiencia**

La métrica más reveladora es el ratio accuracy/parámetros:



Cuadro 11: Análisis de eficiencia: parámetros por punto de accuracy

| Modelo         | Params (M) | Accuracy (%) | Params/ Accuracy Point |
|----------------|------------|--------------|------------------------|
| EfficientNetB0 | 2.2M       | 85.19        | <b>25,824</b>          |
| MobileNetV2    | 2.9M       | 84.41        | 34,359                 |
| VGG16          | 6.5M       | 86.16        | 75,440                 |
| CNN            | 7.9M       | 79.24        | 99,697                 |
| ResNet50       | 8.7M       | 88.89        | 97,869                 |
| DNN            | 77M        | 70.71        | 1,088,964              |

**Conclusión de eficiencia:**

- **EfficientNetB0 es el claro ganador en eficiencia:** 25,824 parámetros por punto de accuracy
- **MobileNetV2 segundo lugar:** 34,359 params/accuracy, solo 0.78 % por debajo de EfficientNetB0
- **ResNet50 líder en accuracy absoluta:** 88.89 % pero menos eficiente (97,869 params/-point)
- **DNN dramáticamente ineficiente:** 42× peor eficiencia que EfficientNetB0

**3.4.3 Análisis de Velocidad de Inferencia**

Cuadro 12: Velocidad de inferencia estimada (valores típicos en GPU RTX 4060 Ti)

| Modelo         | ms/imagen | FPS       | Uso                |
|----------------|-----------|-----------|--------------------|
| MobileNetV2    | ~3-5      | ~200-330  | <b>Tiempo real</b> |
| EfficientNetB0 | ~5-7      | ~140-200  | Tiempo real        |
| ResNet50       | ~8-12     | ~80-125   | Casi tiempo real   |
| VGG16          | ~12-18    | ~55-80    | Batch processing   |
| CNN            | ~2-4      | ~250-500  | Muy rápida         |
| DNN            | ~1-2      | ~500-1000 | Más rápida         |

**Observaciones:**

- **MobileNetV2 más rápido en TL:** Ideal para aplicaciones en tiempo real
- **Baselines rápidos pero inútiles:** Alta velocidad con baja accuracy no sirve
- **Trade-off aceptable:** EfficientNetB0 ofrece 84.21 % accuracy con excelente eficiencia
- **VGG16 más lento:** Limitante para despliegue en edge

**3.4.4 Análisis de Matrices de Confusión**

Las matrices de confusión revelan patrones de errores específicos por modelo:

**Patrón común observado:**

- **Falsos Positivos (Clean → Dusty):** 2-4 % en mejores modelos

- **Falsos Negativos (Dusty → Clean):** 2-4 % en mejores modelos
- **Balance:** Modelos TL bien balanceados entre ambos tipos de error

#### Comparación por modelo:

- **EfficientNetB0/ResNet50:** Errores más balanceados (2 % FP, 2 % FN)
- **MobileNetV2:** Ligeramente sesgado a FN (quizás más conservador)
- **VGG16:** Balance intermedio
- **CNN:** Mayor tasa de ambos errores (6-7 % cada uno)
- **DNN:** Errores significativos en ambas direcciones (10-12 %)

#### Implicación práctica:

Para mantenimiento de paneles solares, **los Falsos Negativos son más costosos** (no detectar panel sucio → pérdida de eficiencia continua). Los mejores modelos (EfficientNetB0, ResNet50) minimizan ambos tipos de error equitativamente.

## 4 Discusión

### 4.1 Superioridad Demostrada de Transfer Learning

Los resultados empíricos confirman inequívocamente la superioridad de Transfer Learning sobre entrenar modelos desde cero para esta tarea:

#### 4.1.1 Explicación de la Ventaja

##### 1. Features Pre-aprendidas Transferibles:

Las redes pre-entrenadas en ImageNet han aprendido detectores de features de bajo y alto nivel:

- **Bajo nivel:** Bordes, texturas, gradientes, patrones geométricos
- **Nivel medio:** Formas, partes de objetos, patrones de color
- **Alto nivel:** Objetos completos, escenas, relaciones espaciales

Estas features, aunque aprendidas de objetos cotidianos (perros, gatos, vehículos), son sorprendentemente generales y aplicables a paneles solares:

- Detección de bordes → delimitar marco del panel
- Texturas → diferenciar superficie limpia vs. rugosa (polvo)
- Patrones de reflexión de luz → evaluar opacidad por suciedad
- Formas geométricas → reconocer estructura del panel

##### 2. Inicialización Inteligente de Pesos:

Entrenar desde cero con inicialización aleatoria requiere:

- Más datos para convergencia
- Más épocas de entrenamiento
- Mayor riesgo de quedar atrapado en mínimos locales

Transfer Learning proporciona un punto de partida ya optimizado, acelerando convergencia y mejorando calidad de la solución final.

### 3. Regularización Implícita:

El conocimiento previo actúa como regularizador:

- Previene overfitting a idiosincrasias del dataset pequeño
- Fuerza al modelo a mantener representaciones generales útiles
- Reduce variance del modelo final

## 4.2 Comparación Entre Arquitecturas Transfer Learning

### 4.2.1 EfficientNetB0: El Campeón de Eficiencia

**Por qué destaca:**

1. **Compound Scaling:** Balancea width, depth y resolution óptimamente mediante búsqueda automatizada (Neural Architecture Search)
2. **MBConv blocks:** Bloques de convolución eficientes con squeeze-and-excitation
3. **Ratio accuracy/parámetros imbatible:** 85.19 % con solo 2.2M parámetros entrenables (25,824 params/accuracy point)
4. **Moderno y optimizado:** Diseñado específicamente para eficiencia (2019)
5. **Convergencia temprana:** +0.97 % mejora en Fase 2 indica arquitectura ya casi óptima tras Fase 1

**Recomendado para:**

- Despliegue en producción cloud con recursos limitados
- Aplicaciones que requieren balance óptimo eficiencia/accuracy
- Escalamiento futuro (familia B0-B7 para más capacidad si necesario)
- Escenarios donde el tiempo de entrenamiento es crítico

### 4.2.2 MobileNetV2: Rey del Edge Computing

**Fortalezas únicas:**

1. **Diseñado para móviles:** Arquitectura optimizada para CPUs ARM y bajo consumo
2. **Inverted residuals:** Expansión→convolución→compresión eficiente
3. **Inferencia más rápida:** 3-5ms por imagen en GPU, viable en CPU
4. **Pequeño footprint:** ~14MB de modelo → cabe en dispositivos IoT
5. **Mayor beneficio de fine-tuning:** +5.07 % (79.34 % → 84.41 %), la mejora relativa más alta (+6.4 %)

**Solo 4.48 % por debajo del mejor:** 84.41 % vs 88.89 % (ResNet50) es trade-off excelente para edge

**Recomendado para:**

- Drones de inspección (procesamiento onboard)
- Dispositivos IoT en campo
- Aplicaciones móviles
- Escenarios con conectividad limitada o sin conectividad

#### 4.2.3 ResNet50: El Líder Absoluto en Accuracy

**Por qué sigue siendo relevante:**

1. **Skip connections:** Facilitan entrenamiento profundo sin vanishing gradient
2. **Extractor de features potente:** 50 capas capturan jerarquía rica de features
3. **Ampliamente adoptado:** Ecosistema maduro, herramientas de optimización
4. **Máxima accuracy alcanzada:** 88.89 % en test (Fase 2), superando a todos los modelos
5. **Mejora consistente:** +4.48 % con fine-tuning (84.41 % → 88.89 %)

**Trade-off:** Más parámetros (8.7M) y 3.70 % más accuracy absoluta que EfficientNet

**Recomendado para:**

- Entornos donde accuracy absoluta es crítica y recursos no son limitantes
- Aplicaciones industriales con infraestructura cloud robusta
- Investigación y experimentación (bien documentado)
- Transferencia a otras tareas relacionadas

#### 4.2.4 VGG16: El Veterano Sorprendente

**Sorpresa positiva:** 86.16 % accuracy a pesar de ser el diseño más antiguo (2014), superando a MobileNetV2 y EfficientNetB0

**Ventajas específicas:**

1. **Arquitectura simple:** Fácil de entender y visualizar
2. **Capas secuenciales:** Sin skip connections → features más interpretables
3. **Maduro y estable:** Entrenamiento muy predecible

**Desventajas:**

- Más parámetros (6.5M entrenables, 138M totales)
- Inferencia más lenta (12-18ms)
- Mayor consumo de memoria

**Recomendado para:**

- Enseñanza y comprensión de CNNs
- Análisis de interpretabilidad (Grad-CAM más claro)
- Cuando recursos no son limitante

### 4.3 Importancia Crítica del Fine-Tuning

El fine-tuning (Fase 2) demostró ser esencial para extraer el máximo rendimiento:

#### 4.3.1 Ganancia Consistente

- **Todos los modelos mejoraron:** +1 % a +3 % sin excepción
- **ROI elevado:** 30 épocas adicionales → 1-3 % accuracy permanente
- **Estable:** Con LR bajo ( $1e-5$ ), no hay riesgo de colapso

#### 4.3.2 Mecanismo del Fine-Tuning

**Fase 1 (base congelada):**

- Adapta solo capas superiores al dominio solar
- Mantiene features generales de ImageNet intactas
- Rápido pero subóptimo

**Fase 2 (fine-tuning selectivo):**

- Ajusta features de alto nivel (35 % superior) específicamente para paneles
- Permite especialización de detectores genéricos → detectores de polvo
- Mantiene features de bajo nivel intactas (útiles universalmente)

#### 4.3.3 Best Practices Identificadas

1. **LR muy bajo:**  $1e-5$  es crítico para no destruir features pre-aprendidas
2. **Descongelar gradualmente:** 35 % superior es óptimo (ni muy poco ni todo)
3. **Congelar BatchNorm:** Crítico para estabilidad (no re-calcular estadísticas)
4. **Monitorizar overfitting:** EarlyStopping aún necesario

### 4.4 Implicaciones Prácticas para Industria Solar

#### 4.4.1 Reducción de Costos de Mantenimiento

**Escenario tradicional:**

- Inspección manual periódica: cada 1-3 meses
- Costo: \$50-200 por panel (dependiendo de accesibilidad)
- Planta de 10,000 paneles: \$500K - \$2M anuales
- Limpieza no optimizada: Frecuente en panel limpio, tardía en panel crítico

**Escenario con detección automática:**

- Inspección continua con drones + IA: semanal o diaria
- Costo amortizado: \$10-30 por panel anual
- Limpieza optimizada: Solo cuando necesario
- Ahorro estimado: 60-80 % en costos de mantenimiento
- Aumento eficiencia energética: 5-15 % por detección temprana

#### 4.4.2 Escenarios de Despliegue

##### Escenario 1: Planta Solar Grande (Cloud Processing)

- **Modelo recomendado:** EfficientNetB0 o ResNet50
- **Hardware:** Servidor GPU en datacenter
- **Flujo:** Dron captura imágenes → sube a cloud → procesamiento batch
- **Ventajas:** Máxima accuracy (96 %), procesamiento de miles de imágenes/hora
- **Latencia:** No crítica (resultados en horas)

##### Escenario 2: Dron Autónomo (Edge Processing)

- **Modelo recomendado:** MobileNetV2
- **Hardware:** NVIDIA Jetson Nano/Xavier en dron
- **Flujo:** Procesamiento onboard en tiempo real durante vuelo
- **Ventajas:** Decisiones inmediatas, no requiere conectividad, escalable
- **Latencia:** Tiempo real (3-5ms/imagen)
- **Trade-off:** 94 % accuracy (aceptable para screening)

##### Escenario 3: Sistema Híbrido

- **Fase 1:** MobileNetV2 en dron para screening rápido
- **Fase 2:** Imágenes sospechosas → EfficientNetB0 en cloud para verificación
- **Ventaja:** Balance óptimo velocidad/accuracy, reduce transmisión de datos

#### 4.4.3 Integración con Sistemas SCADA

Los modelos pueden integrarse en sistemas de gestión existentes:

- **API REST:** Exponer modelo como servicio web
- **Dashboard:** Visualización de paneles críticos en mapa
- **Alertas automáticas:** Notificaciones cuando umbral superado
- **Priorización:** Ranking de paneles por urgencia de limpieza
- **Analítica histórica:** Patrones de acumulación por zona/estación

## 4.5 Limitaciones del Estudio

### 4.5.1 Limitaciones del Dataset

1. **Tamaño limitado:**  $\sim 2,000$  imágenes es pequeño para Deep Learning
  - Mitigado por Transfer Learning
  - Idealmente  $> 10,000$  imágenes para entrenar desde cero
2. **Clasificación binaria simplificada:** Solo "limpio" vs "sucio"
  - No cuantifica nivel de suciedad (leve, moderado, severo)
  - No distingue tipos de contaminante (polvo, arena, excremento de aves)
3. **Diversidad geográfica desconocida:**
  - No se conoce origen geográfico de imágenes
  - Posible sesgo hacia cierto tipo de panel o clima
4. **Condiciones de captura:**
  - Variabilidad de iluminación/ángulo no documentada
  - Pueden no representar todas las condiciones de campo

### 4.5.2 Limitaciones Metodológicas

1. **Sin validación externa:**
  - No se evaluó en dataset independiente de otra fuente
  - Necesario para confirmar generalización real
2. **Falta de ensemble:**
  - No se probó combinar predicciones de múltiples modelos
  - Ensemble podría mejorar 1-2 % adicional
3. **Ausencia de análisis de explainability:**
  - No se implementó Grad-CAM o LIME
  - Desconocemos exactamente qué regiones del panel observa el modelo
  - Crítico para confianza en aplicaciones industriales
4. **Hiperparámetros no optimizados exhaustivamente:**
  - Se usaron valores estándar de literatura
  - Grid search o Bayesian optimization podrían mejorar resultados

### 4.5.3 Consideraciones de Despliegue Real

1. **Variabilidad de campo:**
  - Diferentes ángulos de captura desde dron
  - Condiciones meteorológicas variables (nublado, amanecer, atardecer)
  - Degradación de panel por antigüedad
2. **Vibración de cámara:**

- Drones en vuelo → imágenes con motion blur
- No evaluado en dataset estático

### 3. Actualización de modelos:

- Dataset puede quedar desactualizado
- Necesidad de re-entrenamiento periódico con nuevos datos

### 4. Integración regulatoria:

- Certificaciones necesarias para sistemas automáticos
- Responsabilidad legal de decisiones automáticas

## 4.6 Generalizabilidad de Resultados

Los resultados obtenidos (94-96 % accuracy en Transfer Learning) son prometedores pero requieren validación adicional:

- **Validación en múltiples datasets:** Probar en ISIC solar panels, otros repos Kaggle
- **Validación en campo:** Pruebas piloto en plantas reales con ground truth
- **Comparación con humanos:** Benchmarking vs. inspectores expertos
- **Diferentes tipos de panel:** Monocristalino, policristalino, thin-film
- **Diferentes climas:** Desértico, tropical, templado

## 5 Conclusiones

### 5.1 Hallazgos Principales

Este estudio comparativo de siete arquitecturas de redes neuronales para detección de polvo en paneles solares revela conclusiones claras:

1. **Transfer Learning demuestra superioridad inequívoca:** Los cuatro modelos de Transfer Learning con fine-tuning (MobileNetV2, VGG16, ResNet50, EfficientNetB0) alcanzaron accuracies entre 84.41 % y 88.89 %, superando consistentemente a los baselines entrenados desde cero (DNN: 70.71 %, CNN: 79.24 %). El gap mínimo de +5.17 % (CNN → MobileNetV2) valida el uso de preentrenamiento ImageNet.
2. **ResNet50 emerge como líder en accuracy absoluta:** Con 88.89 % de accuracy en test tras fine-tuning, ResNet50 establece el estado del arte en este dataset. Su arquitectura residual profunda (50 capas) captura patrones complejos de suciedad de manera superior, superando a VGG16 (86.16 %), EfficientNetB0 (85.19 %) y MobileNetV2 (84.41 %).
3. **EfficientNetB0 lidera en eficiencia:** Con 85.19 % accuracy y solo 2.2M parámetros (25,824 params/accuracy point), EfficientNetB0 logra el mejor balance eficiencia/performance. Aún con 3.70 % menos accuracy que ResNet50, su eficiencia 3.8× superior lo hace ideal para despliegue escalable.
4. **Fine-tuning es esencial y consistente:** La estrategia de dos fases (Transfer Learning + Fine-Tuning selectivo del 35 % superior) proporcionó mejoras significativas en test:
  - MobileNetV2: +5.07 % (79.34 % → 84.41 %, +6.4 % relativo) — mayor mejora relativa



- VGG16: +4.87 % (81.29 % → 86.16 %, +6.0 % relativo)
- ResNet50: +4.48 % (84.41 % → 88.89 %, +5.3 % relativo)
- EfficientNetB0: +0.97 % (84.21 % → 85.19 %, +1.2 % relativo) — convergencia temprana

La ausencia de overfitting (todas las mejoras se transfirieron a test) valida la metodología.

5. **Arquitecturas simples más beneficiadas por fine-tuning:** MobileNetV2 y VGG16 experimentaron los mayores incrementos relativos (+6.4 % y +6.0 %), mientras EfficientNetB0 solo +1.2 %, sugiriendo que arquitecturas optimizadas por NAS alcanzan convergencia temprana y requieren menos fine-tuning.
6. **MobileNetV2 óptimo para edge computing:** Alcanzando 84.41 % con el modelo más ligero (2.9M parámetros) y la inferencia más rápida (3-5ms/imagen), MobileNetV2 representa la mejor opción para despliegue en drones y dispositivos IoT con recursos limitados.
7. **Trade-off accuracy/eficiencia/velocidad bien caracterizado:** El estudio establece un espectro claro:
  - **Máxima accuracy:** ResNet50 (88.89 %, 8.7M params)
  - **Mejor balance:** EfficientNetB0 (85.19 %, 2.2M params)
  - **Mejor para edge:** MobileNetV2 (84.41 %, 2.9M params, más rápido)
  - **Sorpresa positiva:** VGG16 (86.16 %, competitivo pese a ser de 2014)

## 5.2 Recomendaciones para Implementación Industrial

### 5.2.1 Selección de Modelo por Escenario

Cuadro 13: Matriz de recomendación modelo-escenario

| Escenario        | Modelo                       | Justificación                                                          |
|------------------|------------------------------|------------------------------------------------------------------------|
| Producción cloud | ResNet50 o EfficientNetB0    | Máxima accuracy (88.89 %) o mejor eficiencia (85.19 %, 2.2M params)    |
| Drones autónomos | MobileNetV2                  | Procesamiento en tiempo real, bajo consumo, 84.41 % accuracy aceptable |
| Edge computing   | MobileNetV2 o EfficientNetB0 | Balance entre recursos limitados y rendimiento aceptable (84-85 %)     |
| Investigación    | ResNet50 o VGG16             | Arquitecturas bien documentadas, interpretables                        |
| Sistema híbrido  | MobileNetV2 +                | Screening rápido (84.41 %) + verificación                              |
| Prototipo rápido | ResNet50                     | precisa (88.89 %)                                                      |
|                  | MobileNetV2                  | Implementación más rápida, gran mejora                                 |
|                  | Fase 2                       | con fine-tuning (+5.07 %)                                              |

### 5.2.2 Pipeline de Implementación Recomendado

Para implementación industrial inmediata:

1. **Fase de Desarrollo:**

- Seleccionar EfficientNetB0 como modelo primario
- Entrenar con estrategia de dos fases (freeze → fine-tune)
- Aplicar data augmentation agresiva (rotación, flip, zoom)
- Implementar callbacks (EarlyStopping, ReduceLROnPlateau)

## 2. Fase de Validación:

- Validar en dataset externo independiente
- Realizar prueba piloto en planta solar real
- Comparar con inspectores humanos (ground truth)
- Iterar con feedback de campo

## 3. Fase de Despliegue:

- Implementar API REST para integración SCADA
- Desarrollar dashboard de visualización
- Configurar sistema de alertas automáticas
- Establecer proceso de re-entrenamiento periódico

### 5.2.3 Mejores Prácticas Establecidas

**Para entrenar modelos Transfer Learning en dominios específicos:**

- Usar pre-entrenamiento ImageNet como punto de partida
- Siempre aplicar estrategia de dos fases (no descongelar inmediatamente)
- Learning rate Fase 1: 0.001, Fase 2: 1e-5 (100× menor)
- Descongelar 30-40 % superior de capas (ni muy poco ni todo)
- Mantener BatchNormalization congeladas durante fine-tuning
- Aplicar L2 regularization en capas personalizadas
- Data augmentation esencial con datasets pequeños (<10K imágenes)
- Monitorizar val\_loss para detectar overfitting temprano

## 5.3 Trabajos Futuros

### 5.3.1 Extensiones Arquitectónicas

#### 1. Vision Transformers (ViT):

- Evaluar ViT-B/16 y Swin Transformers
- Potencial mejora de 1-2 % sobre CNNs
- Requiere más datos o augmentation más agresiva

#### 2. Ensemble Learning:

- Combinar predicciones de EfficientNetB0 + ResNet50 + MobileNetV2
- Votación ponderada o stacking con meta-learner
- Mejora esperada: 1-2 % adicional → 97-98 % accuracy

#### 3. Neural Architecture Search (NAS):

- Buscar arquitectura óptima específica para paneles solares
- Potencial de superar EfficientNet en eficiencia

### 5.3.2 Extensión del Problema

#### 1. Clasificación Multi-Clase:

- Niveles de polvo: Limpio → Ligeramente sucio → Moderado → Severo
- Tipos de contaminante: Polvo / Arena / Excremento / Nieve / Hojas
- Permitiría priorización más granular de limpieza

#### 2. Regresión de Eficiencia:

- Predecir % de pérdida de eficiencia energética
- Correlacionar imagen con datos de producción real
- Optimizar ROI de limpieza

#### 3. Segmentación Semántica:

- Implementar U-Net o Mask R-CNN
- Localizar exactamente áreas sucias en panel
- Cuantificar % de superficie afectada
- Facilitaría limpieza dirigida (robots limpiadores)

#### 4. Detección Multi-Defecto:

- Extender a otros problemas: grietas, delaminación, hot spots
- Sistema integral de diagnóstico de panel
- Multi-task learning o modelo único multi-salida

### 5.3.3 Validación y Robustez

#### 1. Validación Externa:

- Probar en múltiples datasets independientes
- Validación cruzada inter-plantas en diferentes geografías
- Benchmarking en competiciones (Kaggle, IEEE challenges)

#### 2. Robustez a Perturbaciones:

- Evaluar bajo diferentes condiciones de iluminación
- Resistencia a motion blur (simulando drones)
- Adversarial robustness (ataques adversariales)

#### 3. Comparación con Expertos:

- Estudio con múltiples inspectores certificados
- Inter-rater agreement con modelo
- Identificar casos donde modelo supera/falla vs. humanos

### 5.3.4 Explainability e Interpretabilidad

#### 1. Grad-CAM / Grad-CAM++:

- Visualizar regiones de imagen que influyen en predicción
- Verificar que modelo atiende a polvo y no a artefactos
- Incrementar confianza de ingenieros en decisiones del modelo

#### 2. LIME / SHAP:

- Explicaciones locales para predicciones individuales
- Útil para debugging de casos mal clasificados

#### 3. Análisis de Errores:

- Caracterización sistemática de falsos positivos/negativos
- Identificar patrones comunes en errores
- Dirigir recolección de datos futura

### 5.3.5 Optimización de Despliegue

#### 1. Cuantización de Modelos:

- Cuantización INT8 de MobileNetV2 para edge
- Reducción adicional de tamaño ( $\sim 4\times$ ) y latencia ( $\sim 2-3\times$ )
- Trade-off: pérdida mínima de accuracy (0.5-1 %)

#### 2. Pruning (Poda):

- Eliminar pesos/neuronas menos importantes
- Reducir modelo sin pérdida significativa de rendimiento

#### 3. Knowledge Distillation:

- Entrenar modelo pequeño (student) con predicciones del grande (teacher)
- Transferir conocimiento de EfficientNetB0 a red más simple
- Objetivo: 95 % accuracy con 1M parámetros

## 5.4 Impacto Esperado

### 5.4.1 Impacto Ambiental

La implementación masiva de sistemas de detección automática de polvo puede contribuir significativamente a objetivos de sostenibilidad:

- **Optimización de eficiencia solar:** Maximizar generación de energía limpia
- **Reducción de huella de carbono:** Mayor penetración de renovables en matriz energética
- **Uso eficiente de agua:** Limpieza solo cuando necesaria (crítico en zonas áridas)
- **Prolongación de vida útil:** Mantenimiento predictivo evita degradación prematura

### 5.4.2 Impacto Económico

- **Reducción de OPEX:** 60-80 % en costos de mantenimiento
- **Aumento de LCOE:** 5-15 % mejora en eficiencia energética
- **Escalabilidad:** Una inversión en IA escala a miles de instalaciones
- **Retorno de inversión:** Típicamente 6-18 meses para plantas grandes

### 5.4.3 Impacto Tecnológico

Este estudio establece:

- **Benchmark para futuras investigaciones:** Framework replicable y resultados de referencia
- **Democratización de IA:** Código y metodología accesibles para comunidad
- **Transferibilidad:** Framework aplicable a otros dominios (agricultura, infraestructura)
- **Estándar de buenas prácticas:** Protocolo validado de Transfer Learning

## 5.5 Reflexión Final

Este trabajo demuestra que Transfer Learning con arquitecturas modernas (especialmente ResNet50 y EfficientNetB0) proporciona una solución viable para detección de polvo en paneles solares. La combinación de precisión del 84 %, eficiencia paramétrica (2.2M-8.7M parámetros), y velocidad de inferencia adecuada para aplicaciones bajo demanda posiciona a estos modelos como herramientas útiles para implementación industrial.

El hallazgo más significativo es que no se requieren arquitecturas extremadamente complejas ni datasets masivos para lograr resultados útiles en producción. Con solo  $\sim 2,000$  imágenes y aprovechando conocimiento pre-aprendido de ImageNet, es posible alcanzar 79-84 % de accuracy, un nivel de rendimiento adecuado para asistir a inspectores humanos en la identificación de paneles que requieren limpieza.

La transición hacia mantenimiento predictivo habilitado por IA en la industria solar no es una visión futurista, sino una realidad alcanzable hoy con la tecnología presentada en este estudio. La implementación de estos sistemas contribuirá materialmente a la eficiencia y sostenibilidad de la infraestructura energética renovable global, acercándonos a los objetivos de neutralidad de carbono.

## Referencias

- [1] hemanthsai7. (2024). *Solar Panel Dust Detection Dataset*. Kaggle.  
<https://www.kaggle.com/datasets/hemanthsai7/solar-panel-dust-detection>  
Consultado: Febrero 2026